

Python mission guide:

stress_area & custom detection models

How to produce scoreable stressed-crop polygons, install an ONNX detector, and use it safely in a Python autonomy mission.

When the mission is written in Python (`local_planner` / `skytrack_autonomy`), **the framework does not produce the stressed-crop areas for you**. You have to write the detection result to one specific file. After the mission ends the desktop app collects that file and attaches it to the mission report at `status_summary.extras.stress_area`. If the file is missing, in the wrong place or in the wrong format, the report carries no `stress_area` and the detection part scores nothing.

A complete example lives in this folder: [hackathon_example_v1.py](#) — survey the field, find the yellow crop, write `stress_area`, spray it, land.

Why Python code mode

The AI model in the UI only supports **detection models** (bounding boxes), not **segmentation**. A `stress_area` is defined by the outline (polygon) of the stressed patch, so if you want the real shape instead of a rectangle, use **Python code mode**: run your own model or algorithm, work out the stressed areas in ENU coordinates, and write the polygons yourself.

The path

```
/root/.ros/captures/stress_area.json
```

This is the only path that gets collected. A file written anywhere else is not attached.

Notes:

- Create the parent directory yourself before writing (`parents=True`, `exist_ok=True`).
- There must be exactly **one** file named `stress_area.json` under `/root/.ros/captures`, subdirectories included. Do not leave backups or drafts with that name anywhere in there.
- Write **atomically**: write to a temp file under a different name (for example `.stress_area.json.tmp`), then `replace` it onto the real name. That way a mission stopped mid-write leaves the previous, still valid file instead of a truncated one.
- You may write several times during a mission, for instance after each area. Whatever is on disk when the mission ends is what gets collected.
- **Write before landing** — right after the survey stage, for example. Never after `land()`.

Format

```
{
  "stress_area": [
    {
      "class": "stressed",
      "polygon": [[x1, y1], [x2, y2], [x3, y3]]
    }
  ]
}
```

JSON

Field	Type	Meaning
<code>stress_area</code>	list	The areas you found; <code>[]</code> when there are none
<code>class</code>	string	Area label, use <code>"stressed"</code> . Missing means <code>"stressed"</code>
<code>polygon</code>	list of <code>[x, y]</code>	The outline, in world ENU metres (<code>x</code> east, <code>y</code> north), at least 3 vertices

Also:

- The top level **must** be the object `{"stress_area": [...]}`. Do not write a bare list, and do not add other keys.
- The file must be valid JSON, UTF-8 encoded.
- A polygon does **not** need to repeat its first vertex at the end; the scorer closes the ring.
- Coordinates are world ENU, not NED and not image pixels. The pose is NED, so convert: `east, north = pose.y, pose.x`.
- Polygons with fewer than 3 vertices are dropped. Filter out tiny areas yourself to avoid noise.

Where this happens in the example

File: `hackathon_example_v1.py`

- 1 The path, declared once: [line 38](#)

```
STRESS_AREA_PATH = Path("/root/.ros/captures/stress_area.json") # attached to the report by the app
```

PYTHON

- 2 The writer — atomic, and the shape of the file: [lines 110-119](#)

```
def save_stress_areas(areas: list[dict[str, Any]]) -> None:
    STRESS_AREA_PATH.parent.mkdir(parents=True, exist_ok=True)
    tmp = STRESS_AREA_PATH.with_name(".stress_area.json.tmp")
    tmp.write_text(json.dumps({"stress_area": areas}, separators=(",", ":")))
    tmp.replace(STRESS_AREA_PATH)
```

PYTHON

- 3 Called after the survey, **before landing**: [lines 82-83](#)

```
areas = stress_map.stress_areas()
save_stress_areas(areas)
```

PYTHON

- 4 Where each entry comes from: [lines 177-189](#). `StressMap.stress_areas()` returns `[{"class": "stressed", "polygon": [[x, y], ...]}, ...]` with `x, y` already converted to ENU metres.

Replace `StressMap` with your own detection, but keep the path, the format and the way the file is written.

Using your own detection model

The UI's AI model list is fixed. To detect with a model of your own — or with the sample one — install it from your Python mission. A ready-made sample ships next to this guide: [sample_ai_model.zip](#).

File in the zip	What it is
<code>sample_ai_model/det-h2026-v26n-b-fp32-640.onnx</code>	Detector for stressed crop, 640×640, fp32
<code>sample_ai_model/sample.json</code>	Its catalogue entry: one dataset class, <code>stressed</code>

Bring your own model in the same shape:

- **ONNX, end-to-end export.** Output `[1, N, 6]` = `(x1, y1, x2, y2, score, class_id)` in input pixels. The detector runs no NMS of its own.

- **The file is named after the model id:** `<model id>.onnx`, where the id is the key under `models` in the JSON.
- **classes order is class_id order:** `classes[0]` is class id 0, and so on.
- **Ask for a class by its exact name.** The `classes` you request must be names from the dataset's `classes` (case does not matter). `object_mapping` is only a hint for the UI.

1. Copy the zip into the drone container

From a terminal on your machine, with the simulation running:

```
C=skytrack-simulation-skytrack-autonomy-1
docker exec $C mkdir -p /var/www/files/ai_models
docker cp sample_ai_model.zip $C:/var/www/files/ai_models/
```

SHELL

`/var/www/files` is a Docker volume, so the zip survives the app recreating the containers between missions. Anything outside it — `/opt/skytrack/ai` included — is reset each time, which is why the mission installs the model again every run.

2. Install it when the mission starts

```
import json
import zipfile
from pathlib import Path

AI_ROOT = Path("/opt/skytrack/ai") # the detector's catalogue
MODEL_ZIP = Path("/var/www/files/ai_models/sample_ai_model.zip") # where you docker cp'd it

def install_model(zip_path: Path = MODEL_ZIP, ai_root: Path = AI_ROOT) -> list[str]:
    """Copy the zip's ONNX weights into the catalogue and register them. Returns the model ids."""
    with zipfile.ZipFile(zip_path) as archive:
        names = [n for n in archive.namelist() if not n.startswith("__MACOSX/")]
        manifest = json.loads(archive.read(next(n for n in names if n.endswith(".json"))))
        for model_id in manifest["models"]:
            weights = next(n for n in names if n.endswith(f"/{model_id}.onnx") or n == f"{model_id}.onnx")
            target = ai_root / "common" / f"{model_id}.onnx"
            target.parent.mkdir(parents=True, exist_ok=True)
            target.write_bytes(archive.read(weights))

    mapping_path = ai_root / "mapping.json"
    catalogue = json.loads(mapping_path.read_text())
    for section in ("dataset", "models", "object_mapping"):
        catalogue.setdefault(section, {}).update(manifest.get(section, {}))
    tmp = mapping_path.with_name(".mapping.json.tmp") # atomic: the detector re-reads it live
    tmp.write_text(json.dumps(catalogue, indent=4))
    tmp.replace(mapping_path)
    return list(manifest["models"])
```

PYTHON

The built-in models stay available, and running it twice is harmless. The detector picks up the new catalogue on its next request; nothing needs restarting.

3. Detect with it

Install before the drone boots, then register the `Detector` service with your model:

```
from local_planner import boot_drone
from skytrack_autonomy import Detector

MODEL_NAME = "det-h2026-v26n-b-fp32-640"
CLASSES = ["stressed"]

def main() -> None:
    install_model()
    with boot_drone() as drone:
        drone.add_service(Detector(model_name=MODEL_NAME, classes=CLASSES))
        drone.fly(my_mission)
        drone.run()
```

PYTHON

Inside the mission, request a detection and read it back once it has answered:

```
detector = ctx.services.detector
before = detector.count
detector.request(model_name=MODEL_NAME, classes=CLASSES, confidence_threshold=0.4)
# ... hover until detector.count > before (the detector never moves the drone) ...
outcome = detector.last_result
for det in outcome.detections:
    print(det.class_name, det.score, det.bbox.center_x, det.bbox.center_y,
          det.bbox.size_x, det.bbox.size_y)
```

Every box comes back with `class_name == "stressed"`. Boxes are in **image pixels** (`outcome.image_width` × `outcome.image_height`), not world coordinates: turning them into `stress_area` polygons is up to you — project them through the nadir camera the same way `StressMap.add_picture` does in [hackathon_example_v1.py](#).